

Course Work Project Description and Rubric

Semester	202510	Division	CIS
Assessment title in Syllabus	Final Project – Deliverable 1 & 2	Program	IS & IT (CIA)
Course Code	CIS 3113		
Course Title	Object-Oriented Analysis and Design		
CLOs	CLO1, CLO2, CLO3, CLO4, CLO5	Accreditation Body	CAA & CIPS
Course Instructor	Dr. Daoud	CRN	202510_11006
Assessment Weight	30%	Submission Date	Week 15

For Group Work submissions, an additional individual assessment will be conducted.
Grades for the students in one group will vary based on individual performance in an additional assessment.

Student Declaration:

Academic Integrity Statement

In accordance with the HCT Academic Integrity Policy

- Students are required to refrain from all forms of academic integrity breaches as defined and explained by HCT.
- A student found guilty of having committed acts of academic integrity breach(es) will be subject to the relevant sanctions as outlined by HCT.

إفادة النزاهة الأكاديمية

وفقاً لسياسة كليات التقنية العليا للنزاهة الأكاديمية

- على الطلبة الالتزام بلوائح وقواعد النزاهة الأكاديمية، كما هو مبين وموضح في السياسات والإجراءات الخاصة بكليات التقنية العليا.
- في حالة ارتكاب الطالب أي شكل من أشكال الإخلال بالنزاهة الأكاديمية، سيتعرض إلى العقوبات الموضحة في السياسات ذات الصلة.

This assignment is entirely my own work except where I have duly acknowledged other sources in the text and listed those sources at the end of the assignment. I have not previously submitted this work to the HCT or any other entity. I understand that I may be orally examined on my submission.

Student (s) Signature:

Student Name(s):	Hessa Khalfan	Maryam Ali
Student HCT ID(s):	H00535166	H00535836

For Examiner's Use Only

Deliverable No.	Report			Q&A / Viva voce	Total	%
	1 (CLO1, CLO2)	2 (CLO3, CLO4)	3 (CLO5)	1		
Marks Allocated	12.5	25	22.5	40	100	30

1. ANALYSIS MODEL

A) Suitable Development Process

For the Patient Management System project, our team has selected an **Agile Development** process, to be implemented using the **Iterative and Incremental model**. This methodology provides the structure and flexibility required to effectively manage a project of this nature and complexity.

Our decision is based on several key principles. The system can be naturally divided into distinct components, such as patient registration, appointment scheduling, and payment processing. Following the iterative model, we plan to **grow the system piece by piece through multiple iterations**. In each iteration, which will be a two-to-four-week Sprint, our team will complete a small part of the system, covering all core processes from design to testing. The primary goal is to produce a **working part of the system** at the end of each iteration, which serves as the main measure of our progress.

Furthermore, this iterative model strongly supports the Agile principle of **emphasizing flexibility to anticipate new requirements**. By developing the system in incremental steps, we create multiple opportunities to validate our design and functionality. Each Sprint Review serves as a checkpoint to ensure our work aligns with the project's objectives. This allows our team to **embrace change incrementally**, adapting to a deeper understanding of the requirements and ensuring that the final product is a high-quality and effective solution.

Finally, this methodology is perfectly supported by our chosen project management tool, **JIRA**. As a tool designed for Agile and Scrum, JIRA will enable us to plan our Sprints and break down work into user stories and tasks. This directly addresses the project's requirement for collaboration to be **recorded, tracked, and monitored**, ensuring clear visibility of each team member's contribution.

b) System Requirements

The system requirements for the Patient Management System are divided into two main categories: functional and non-functional requirements. Functional requirements define the specific activities the system must perform, while non-functional requirements define the characteristics and qualities of the system.

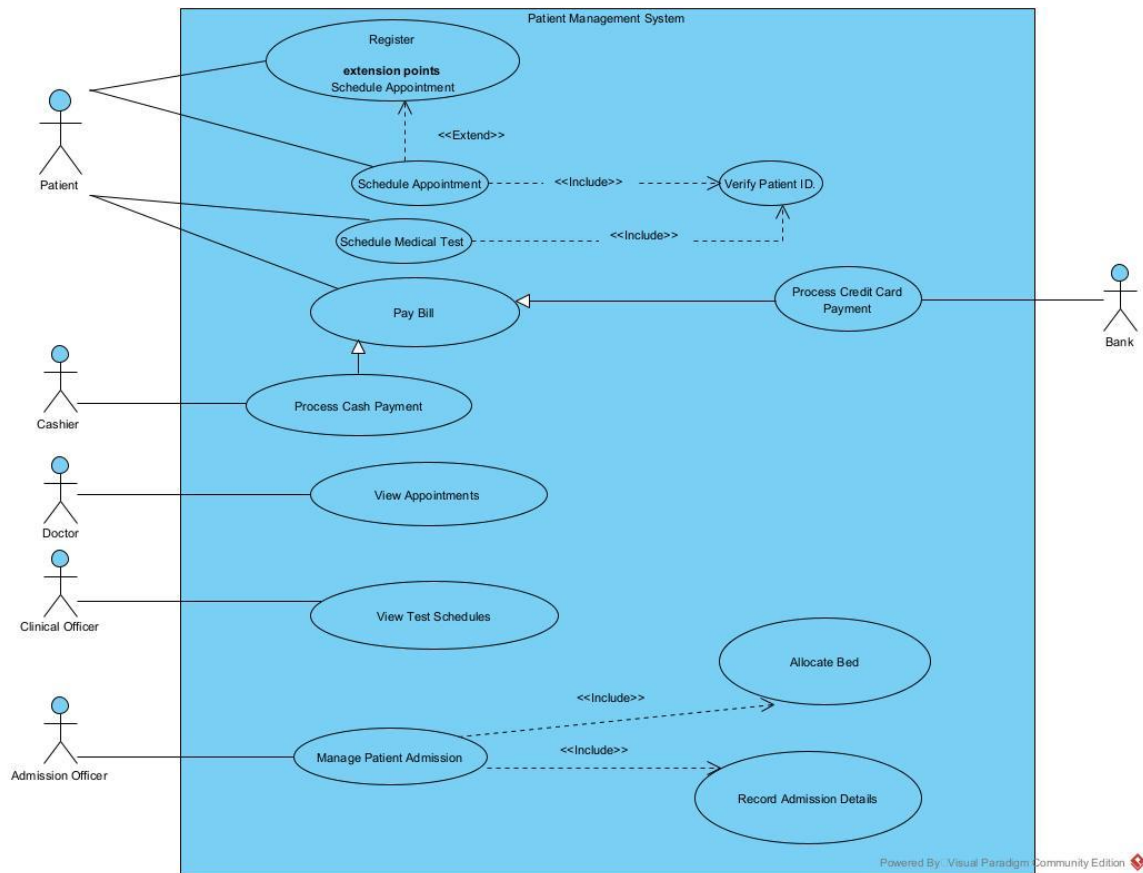
Core Functional Requirements

- **Patient Registration:** A new patient creates their own record in the system by providing essential details. The system assigns a unique patient ID.
- **Appointment Scheduling:** Patient searches for a doctor and selects a preferred time for a consultation.
- **Medical Test Scheduling:** System schedules a medical test for a patient and sends the schedule to the clinical officer.
- **Patient Admission:** Admission Officer manages a patient's admission to the hospital, including assigning a bed and ward.
- **Payment Processing:** Patient pays their bill using either cash or credit card.
 - Cash payments are handled by the Cashier.
 - Credit card payments are processed by the Bank.
- **Automated Notifications:**
 - Appointment confirmation is sent to the patient and doctor upon booking.
 - Test schedule information is sent to the clinical officer.

Business Rules & Non-Functional Requirements:

- Each patient must have a unique ID for identification and tracking.
- A patient must be verified using their ID before they can schedule an appointment.
- The system must check a doctor's availability before confirming an appointment.
- An appointment is tied to exactly one patient and one doctor.
- The payment process moves through three distinct states: Submission, Processing, and Approval.
- Each payment transaction must be recorded with a unique transaction ID, payment type, and amount.
- **Usability:** The system's interface should be intuitive and easy to use, requiring minimal training for hospital staff.
- **Reliability:** The system must maintain high availability during operational hours.
- **Performance:** Key system responses must be completed in under 3 seconds.
- **Security:** Patient data must be confidential and accessible only via secure login.

c) Use Case Diagram and Textual Descriptions



Use Case Diagram Description for the Patient Management System

This Use Case diagram illustrates the interactions between different users (actors) and the Patient Management System. It shows the main functions each user can perform.

Actors and Their Roles:

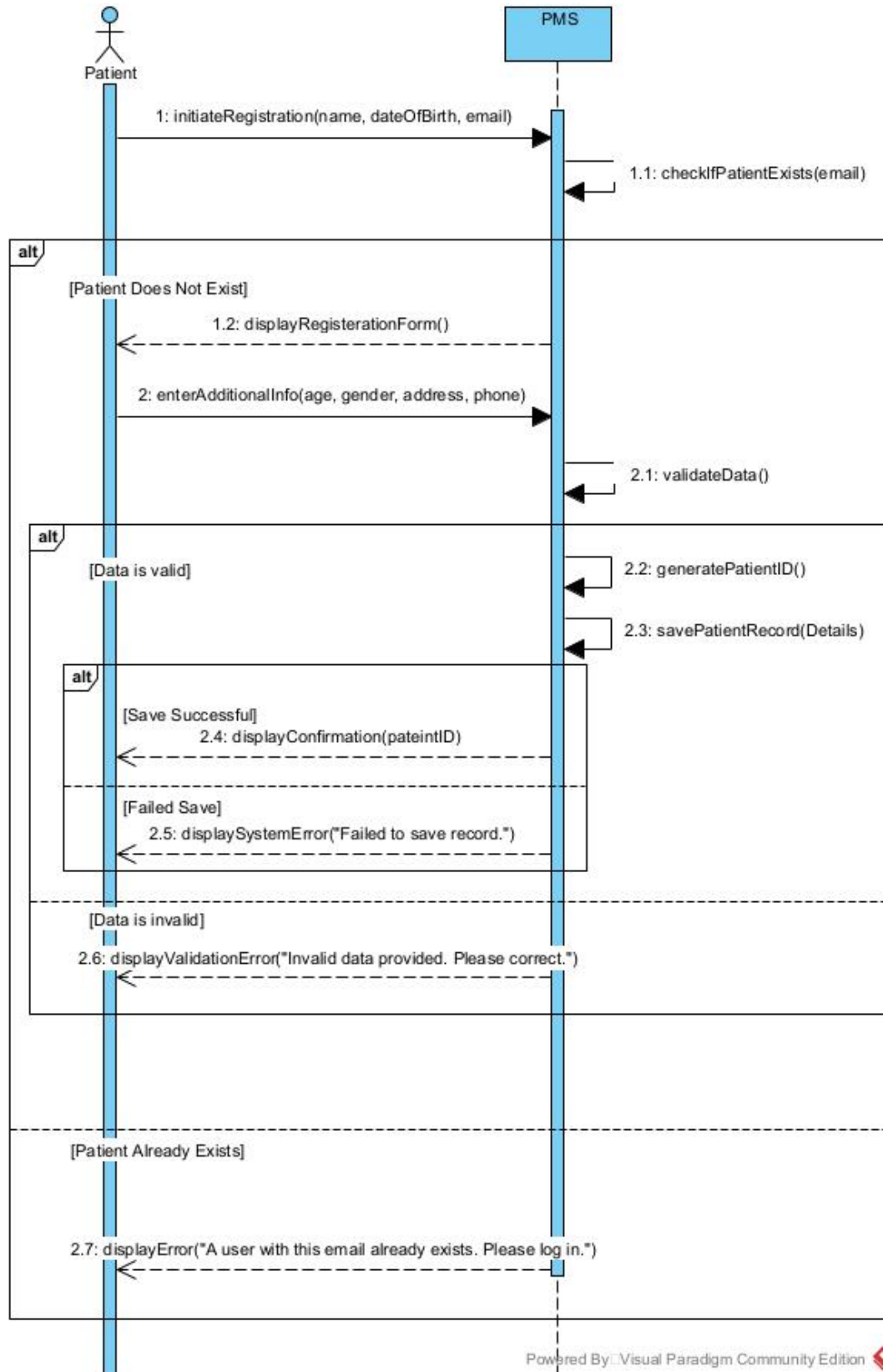
- Patient:** A patient can interact with the system to perform several tasks. They can **Register** as a new patient. They can **Schedule an Appointment** with a doctor and **Schedule a Medical Test**. Both actions require the system to **Verify Patient ID** first. They can **Pay Bill**. This can be done either through cash or credit card.
- Cashier:** The cashier is responsible for handling cash payments. They use the system to **Process Cash Payment**.

- **Bank:** The bank is an external system that processes credit card transactions. It interacts with the system to **Process Credit Card Payment**.
- **Doctor:** The doctor uses the system to manage patient care. They can **View Appointments** to see their schedule.
- **Clinical Officer:** This user is responsible for test schedules. They can **View Test Schedules** to see which tests are planned.
- **Admission Officer:** This user handles the entire patient admission process. They **Manage Patient Admission**. This is a multi-step process that always includes two mandatory sub-tasks:
 1. **Allocate Bed:** Assigning a bed and ward to the patient.
 2. **Record Admission Details:** Saving the admission date and other relevant information.

Key Relationships:

- **<<Include>>:** This shows a mandatory function. For example, Schedule Appointment must **include** Verify Patient ID.
- **<<Extend>>:** This shows an optional function. For example, the Register use case has an extension point for Schedule Appointment, meaning a patient *might* schedule an appointment after registering, but it's not required.
- **Generalization (Arrow with a hollow head):** This shows that one use case is a specialized version of another. For example, Process Cash Payment and Process Credit Card Payment are both specific types of the general Pay Bill process.

Use Case 1: Register New Patient



Actor(s): Patient

Description: This use case allows a new Patient to create a personal account in the system. The process begins after the system prompts a non-registered user to sign up. It validates that the patient is not already in the system, gathers their information, and assigns them a unique Patient ID upon successful completion.

Pre-conditions:

- The user is not logged into the system.
- The system has prompted the user to register (e.g., after a failed search for their ID).

Post-conditions:

- **On Success:** A new patient record is created and stored in the database with a unique Patient ID.
- **Failure:** No new record is created. The system remains in its previous state.

Normal Flow:

1. The Patient initiates the registration process by providing their name, date of birth, and email.
2. The system checks if a user with the provided email already exists.
3. The system confirms the user does not exist and displays the full registration form.
4. The Patient enters their remaining information (age, gender, address, phone number).
5. The system validates that all entered data is complete and in the correct format.
6. The system generates a unique Patient ID.

7. The system saves the complete patient record in the database.
8. The system displays a confirmation message to the Patient, including their new Patient ID.

Alternative Flows:

- **Patient Already Exists:**

1. In step 2, if the system finds that the provided email is already associated with an existing account, it displays an error message (e.g., "A user with this email already exists. Please log in.").
2. The use case terminates.

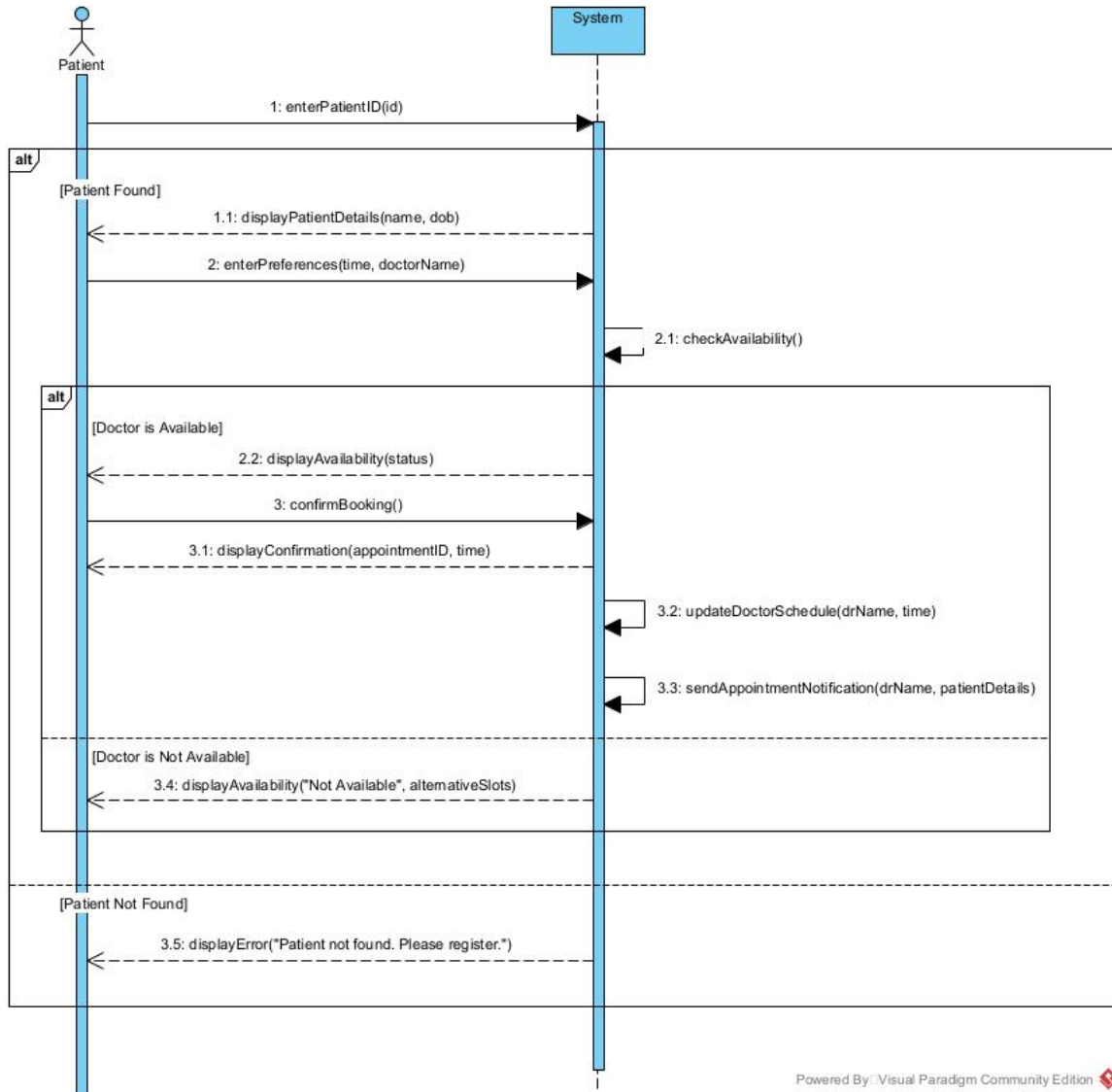
- **Invalid Data:**

1. In step 5, if the system detects that any of the provided information is invalid or missing, it displays an error message prompting the Patient to correct the specific fields.
2. The flow returns to step 4.

- **System Failure:**

1. In step 7, if the system fails to save the new record to the database, it displays a system error message.
2. The use case terminates.

Use Case 2: Schedule Appointment



Actor(s): Patient

Description: This use case allows a registered patient to schedule a new consultation appointment with a doctor. The system verifies the patient's identity, checks the doctor's availability, and confirms the booking.

Pre-conditions:

- The patient must have a registered account in the system.

Post-conditions:

- On Success:** A new appointment record is created and linked to the patient and the doctor. The system internally updates the doctor's schedule and sends notifications.

- **On Failure:** No appointment is made, and the system remains unchanged.

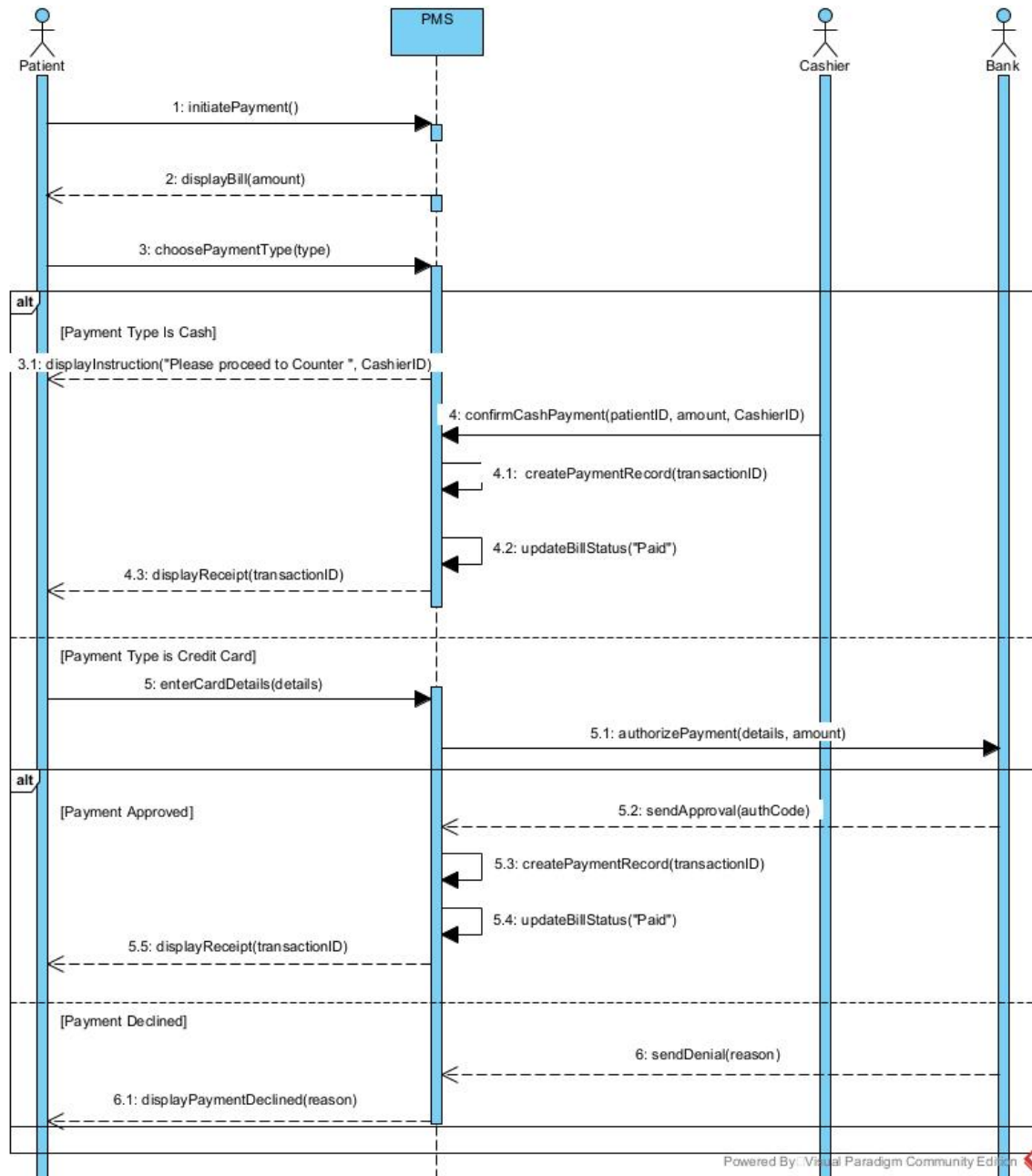
Normal Flow:

1. The Patient initiates the process to schedule an appointment.
2. The system prompts the Patient to enter their Patient ID for verification.
3. The system verifies the Patient ID. Upon successful verification, it displays the doctors' details.
4. The Patient enters their preferred time and the name of the doctor for the consultation.
5. The system internally checks the doctor's schedule for availability at the selected time.
6. The system confirms the doctor is available, creates a new appointment, and generates a unique Appointment ID.
7. The system displays a confirmation message to the Patient with the final appointment details (ID, time, doctor).
8. The system internally updates the respective doctor's schedule and sends them notification with the appointment details.

Alternative Flows:

- **Patient Not Found:** If the Patient ID entered in step 3 is not found in the system, an error message is displayed, prompting the user to register. The use case terminates.
- **Doctor Not Available:** If the system finds that the doctor is not available at the selected time in step 5, it informs the Patient and may suggest alternative available slots. The flow then returns to step 4 for the Patient to enter a new preference.

Use Case 3: Process Patient Payment



Actor(s): Patient, Cashier, Bank

Description: This use case describes how a Patient settles their bill for services received. The process is initiated by the Patient, and depending on the chosen payment method, involves either the Cashier (for cash payments) or the Bank system (for credit card payments).

Pre-conditions:

- An outstanding bill has been generated for the Patient in the system.

Post-conditions:

- **On Success:** The Patient's bill is marked as "Paid". A new payment record, linked to the transaction, is created and stored.
- **On Failure:** The bill remains unpaid. The system state is unchanged.

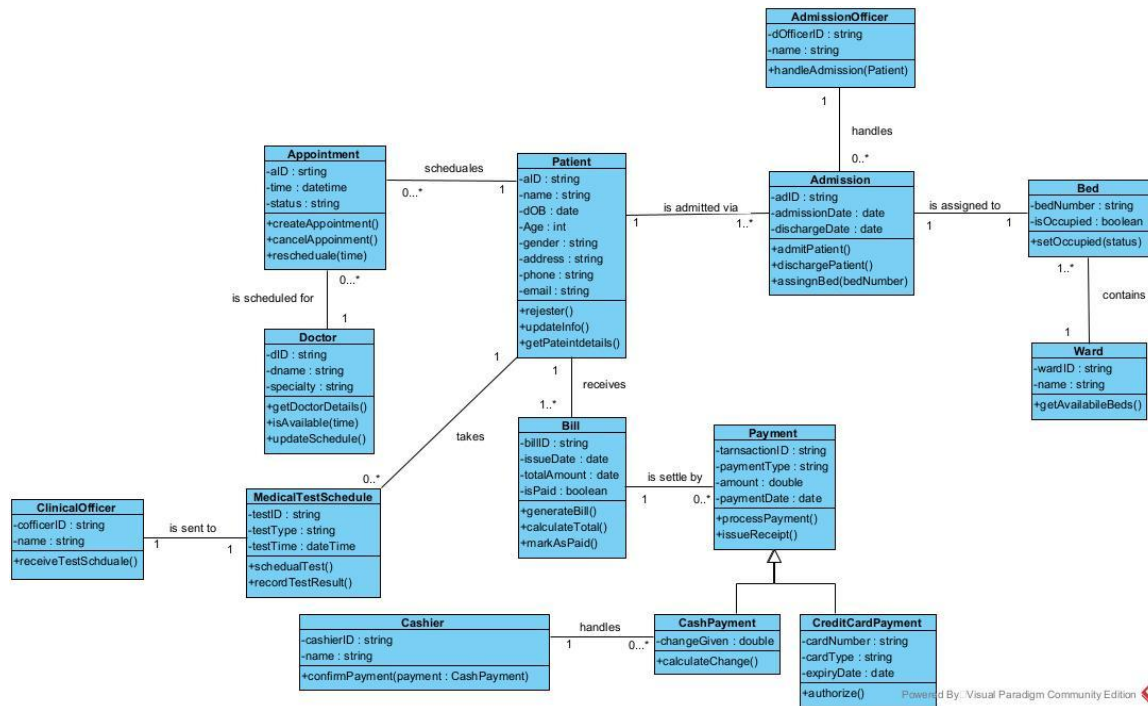
Normal Flow:

1. The Patient initiates the payment process through the system.
2. The system retrieves and displays the patient's outstanding bill amount.
3. The system prompts the Patient to choose a payment type (Cash or Credit Card).
4. **If the payment type is "Cash":**
 - a. The system directs the Patient to a specific cashier, providing their ID or counter number ("Please proceed to Cashier C-102").
 - b. The **Cashier** receives the physical cash from the Patient.
 - c. The **Cashier** confirms the cash payment in the system, referencing the patient's details and the amount.
5. **If the payment type is "Credit Card":**
 - a. The Patient enters their credit card details into the system.
 - b. The system sends an authorization request with the payment details to the external Bank system.
 - c. The Bank system processes the transaction and sends back an approval code.
6. Upon successful confirmation (from either the Cashier or the Bank), the system generates a unique transaction ID and creates a payment record.
7. The system updates the bill's status to "Paid".
8. The system issues a digital or printable receipt to the Patient.

Alternative Flows:

- **Payment Declined (Credit Card):** If the Bank system declines the payment in step 5c, it sends a denial reason. The system then informs the Patient of the decline and may prompt them to try another card or switch to cash payment. The use case may then return to step 3 or terminate.

Domain Class Diagram



Core Classes:

- **Patient:** This is the central class, representing a patient. It stores all personal information like patientID, name, address, and phone. It also includes functions like register() and updateInfo().
- **Appointment:** Represents a single consultation scheduled by a patient. It is linked to one Patient and one Doctor. It holds details like time and status.
- **Doctor:** Represents a doctor in the hospital. It stores their name and specialty and has functions to check availability (isAvailable).
- **MedicalTestSchedule:** Holds information about a medical test scheduled for a patient, including testType and testTime. It is linked to one Patient.
- **Bill:** Represents a patient's bill for services. It is linked to a Patient and contains the totalAmount and a boolean isPaid.
- **Payment:** This class manages the payment process. It has a transactionID and amount. It is a general class, with two specific types:

- **CashPayment:** A specific type of payment made with cash.
- **CreditCardPayment:** A specific type of payment made with a credit card.

Staff and Hospital Structure Classes:

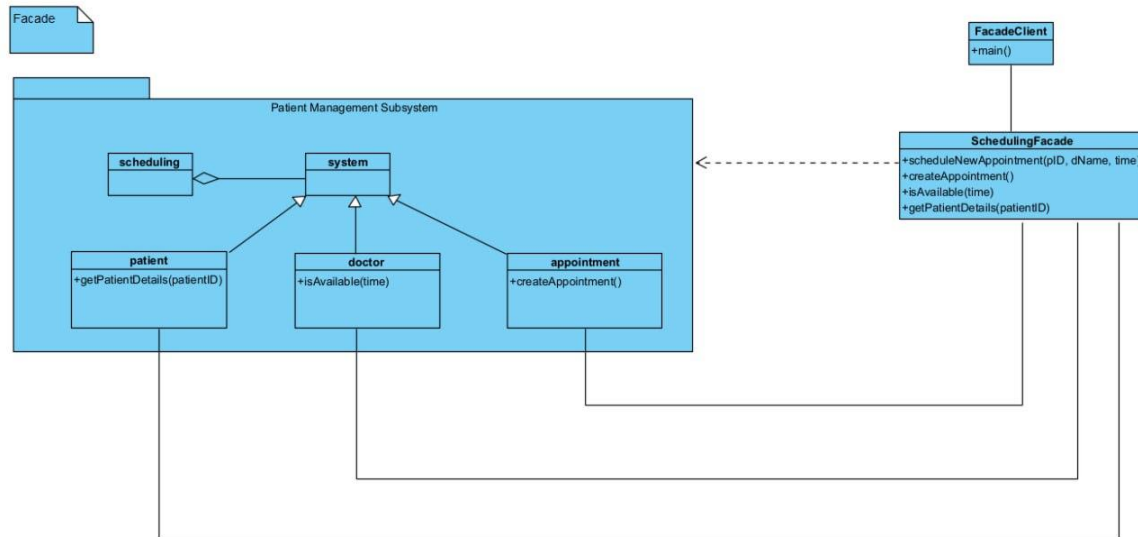
- **AdmissionOfficer:** A staff member who handles the patient admission process.
- **Admission:** Represents the record of a patient's admission to the hospital. It links a Patient to a Bed and includes admissionDate and dischargeDate.
- **Bed:** Represents a single bed in the hospital, which is part of a Ward. It has a status to show if it is isOccupied.
- **Ward:** Represents a hospital ward, which contains one or more beds.
- **Cashier:** A staff member who handles cash payments.
- **ClinicalOfficer:** A staff member who receives information about scheduled medical tests.

Key Relationships:

- A Patient can have zero or more (0..*) Appointments and MedicalTestSchedules.
- An AdmissionOfficer handles zero or more (0..*) Admissions.
- Each Admission is for one Patient and is assigned one Bed.
- A Bill is settled by one or more (0..*) Payments.
- The relationship between Payment, CashPayment, and CreditCardPayment is **Generalization**, meaning cash and credit card payments are specialized types of general payment.

2: HIGH-LEVEL DESIGN AND DESIGN PATTERN

Design Pattern 1: Facade Pattern Diagram



Problem: The process of scheduling appointments is complex. It involves multiple classes working together: first, we need to get the patient's details from the Patient class, then check the doctor's availability from the Doctor class, and finally create a new record in the Appointment class. If the client (like the user interface) has to manage all these steps, the code becomes complicated and tightly coupled to the internal system.

Solution: To solve this, we have implemented the **Facade pattern**. We created a new class called **SchedulingFacade** that acts as a simple, single entry point for the entire scheduling process.

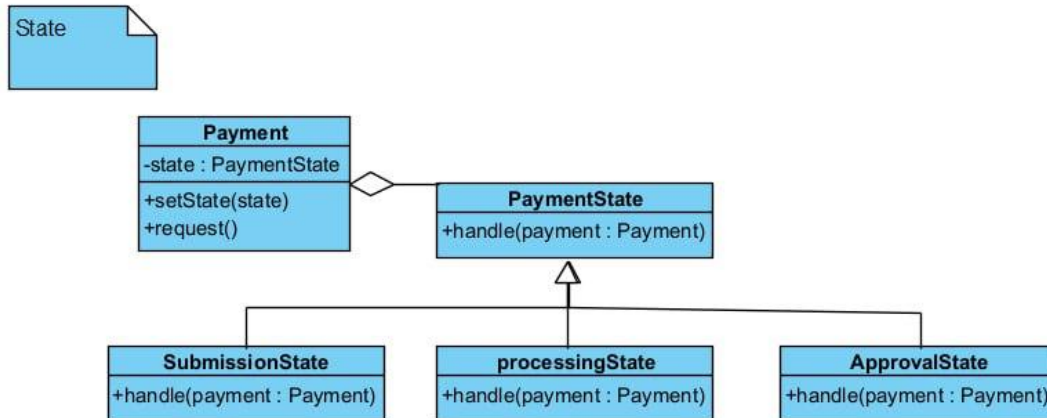
How it is Implemented: As shown in the diagram, the **SchedulingFacade** provides simple methods to the client, such as `scheduleNewAppointment()`. When this method is called, the Facade itself does all the hard work in the background. It communicates with the different classes inside the Patient Management Subsystem in the correct order to perform the task.

The **FacadeClient** (the user) only needs to talk to the **SchedulingFacade** and does not need to know about the complex details of the subsystem.

Benefit: This pattern hides the complexity of the system and provides a clean, simple interface. It makes the system easier to use and reduces the dependency between the

client and the internal classes, which makes our code easier to maintain and update in the future.

Design Pattern 2: State Pattern Diagram



Problem: The case study states that a Payment object goes through three different states: **Submission**, **Processing**, and **Approval**. The behavior of the Payment object changes depending on which state it is in. For example, in the "Processing" state, the system calculates charges, while in the "Approval" state, it verifies the payment. Managing these different behaviors with if/else statements inside the Payment class would make it complex and hard to maintain.

Solution: To manage this, we have implemented the **State pattern**. This pattern allows an object to change its behavior when its internal state changes. The object will appear to change its class.

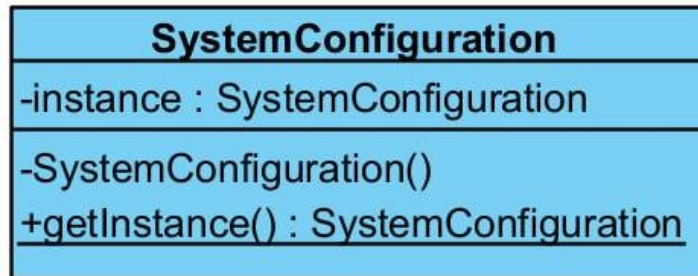
How it is Implemented:

1. We created an abstract class (or interface) called PaymentState with a handle() method.
2. We then created a separate, concrete class for each state: SubmissionState, processingState, and ApprovalState. Each of these classes implements the handle() method in its own specific way, defining the behavior for that particular state.
3. The main Payment class holds a reference to a PaymentState object. Instead of implementing the behavior itself, the Payment class delegates the work to its current state object by calling the handle() method on it.

When the payment process moves to the next stage, we simply change the state object inside the Payment class using the setState() method.

Benefit: This pattern keeps the code clean and organized. Each state's logic is isolated in its own class, making it easy to add new states or change existing ones without touching the Payment class itself. This follows the Open/Closed Principle, making our design more flexible and maintainable.

Design Pattern 3: Singleton Pattern



Problem: In our system, there are certain classes that should only ever have one single instance (one object) created. A good example is a SystemConfiguration class, which holds global settings for the application, like the database connection string or the hospital's name. If we create multiple instances of this class, we could have conflicting settings and waste memory.

Solution: To solve this, we have implemented the **Singleton pattern**. This pattern ensures that a class has only one instance and provides a single, global point of access to it.

How it is Implemented: As shown in the SystemConfiguration class diagram:

1. We created a private static variable called instance to hold the single object of the class.
2. We made the constructor private (-SystemConfiguration()). This is very important because it prevents anyone from creating a new instance of the class from outside.
3. We created a public static method called getInstance(). This is the only way to get the object. The first time this method is called, it creates the new instance and stores it in the instance variable. On every subsequent call, it simply returns the already-existing instance.

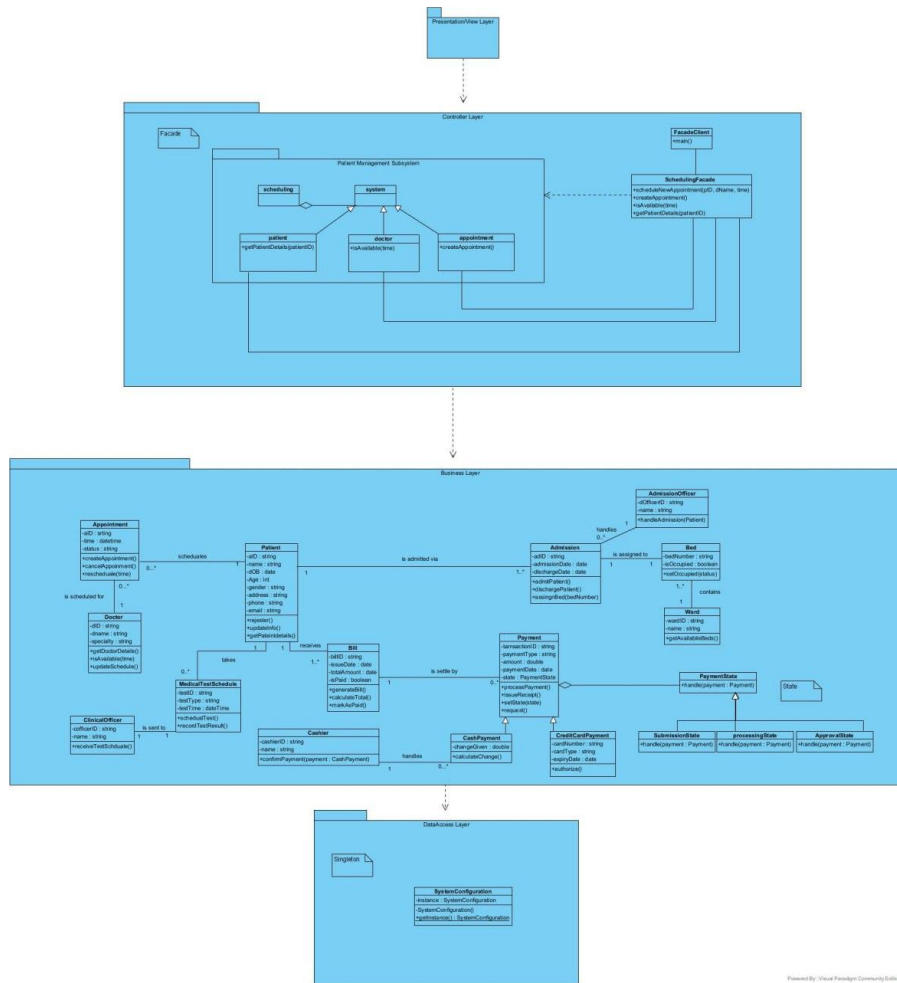
Benefit: This pattern guarantees that there is only one SystemConfiguration object throughout the entire application. This provides a global access point to the

كليات التقنية العلية



The Payment class has been updated to use the **State pattern**, allowing its behavior to change dynamically through different PaymentState objects like SubmissionState, processingState, and ApprovalState.

g) High-Level Architectural Model



This diagram shows our system's 4-layer architecture, which organizes the code for easier maintenance.

1. **Presentation Layer:** The User Interface (UI) that the user interacts with.
2. **Controller Layer:** Acts as a middleman. It takes user requests from the UI (using the **Facade** pattern) and tells the Business Layer what to do.
3. **Business Layer:** The core of the application. It contains all the business logic and rules (our main Class Diagram and the **State** pattern).
4. **Data Access Layer:** Responsible for saving and retrieving all data from the database (using the **Singleton** pattern to manage configuration).

This separation makes the system flexible and easy to update.

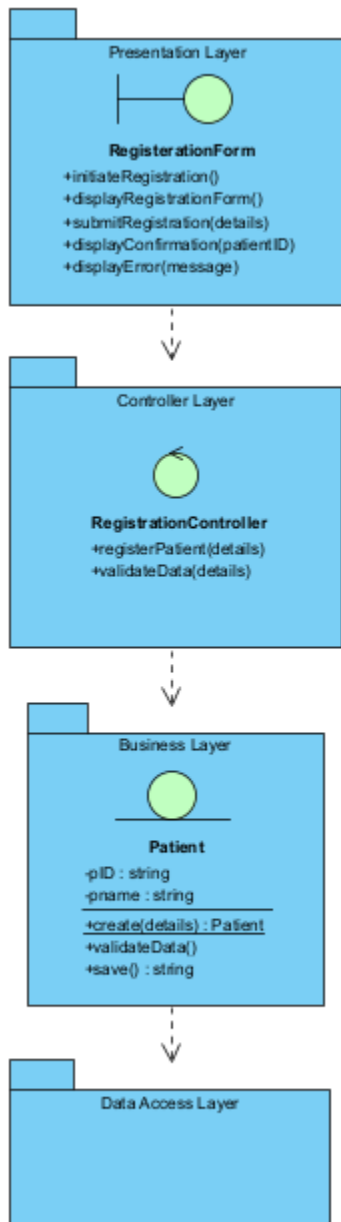
3: DETAILED-LEVEL DESIGN MODEL

Sprint Identification

In this section, we identify three distinct sprints, each corresponding to a core use case from our analysis model. This iterative approach allows our team to focus on developing, designing, and validating a specific piece of functionality in a structured manner. Each sprint is assigned to a team member, ensuring clear ownership and accountability.

- **Sprint 1: New Patient Registration**
 - **Use Case:** Register New Patient
 - **Objective:** To design the complete flow for a new patient to create an account in the system.
- **Sprint 2: Appointment Scheduling**
 - **Use Case:** Schedule Appointment
 - **Objective:** To design the process for a registered patient to find a doctor and book an available appointment slot.
- **Sprint 3: Patient Payment Processing**
 - **Use Case:** Process Patient Payment
 - **Objective:** To design the payment workflow, handling both cash and credit card transactions, and applying the State design pattern.

Sprint 1: New Patient Registration



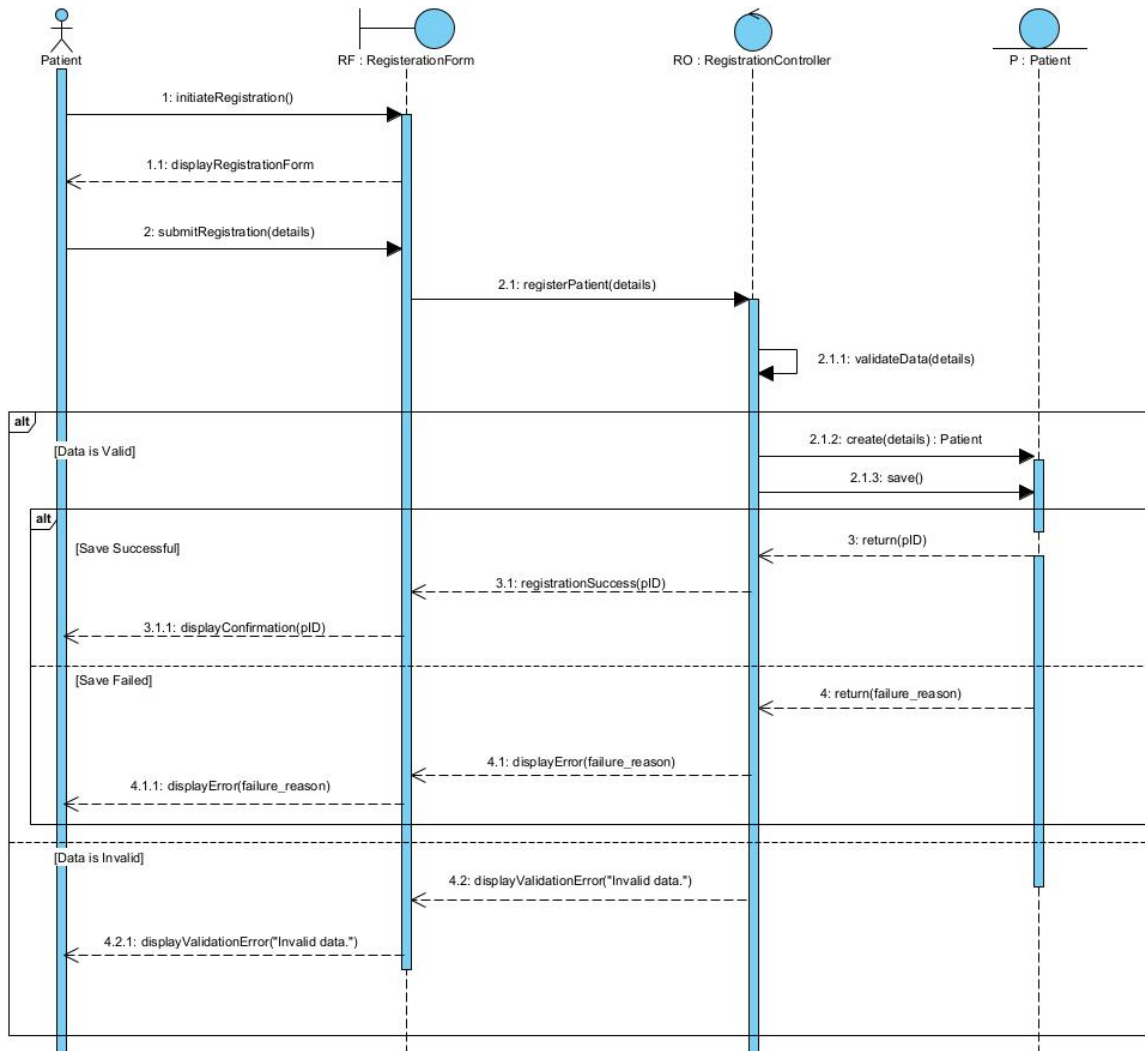
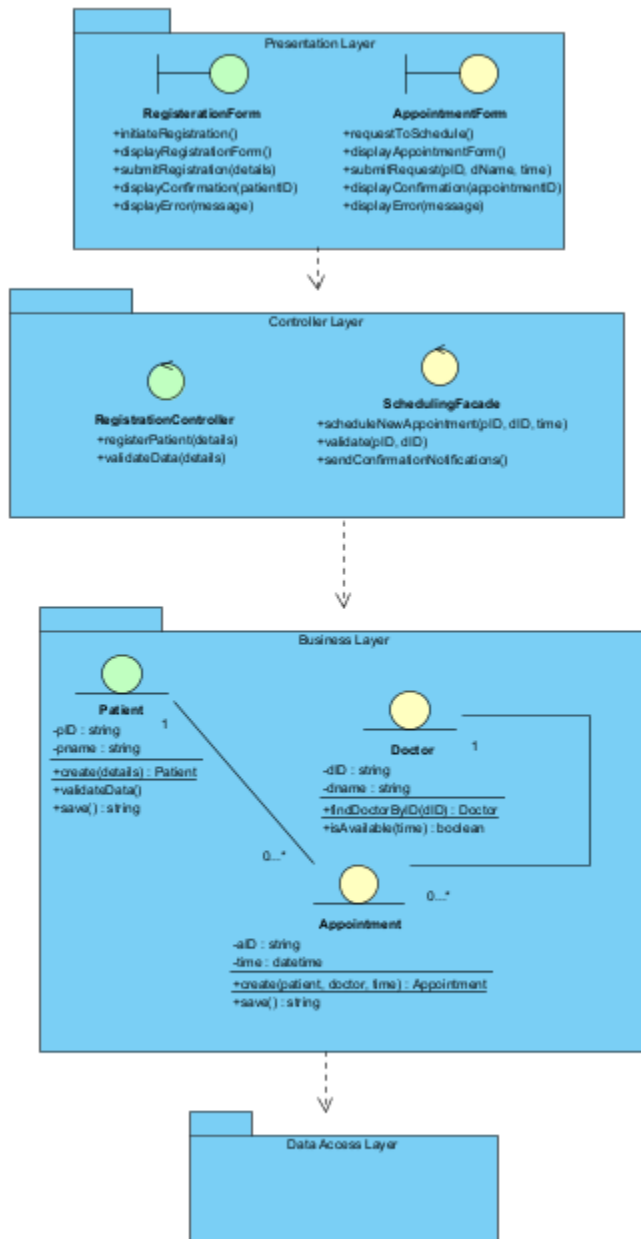


Diagram Description:

This diagram illustrates the detailed interactions for the patient registration process. The flow begins at the RegistrationForm (boundary object), which passes the user's data to the RegistrationController (control object). The controller manages the process by first validating the input (validateData) and then using a static factory method (create) on the Patient entity. Finally, the controller calls the save method on the new Patient object to store it, demonstrating a clear separation of concerns between the controller and the entity.

Sprint 2: Appointment Scheduling



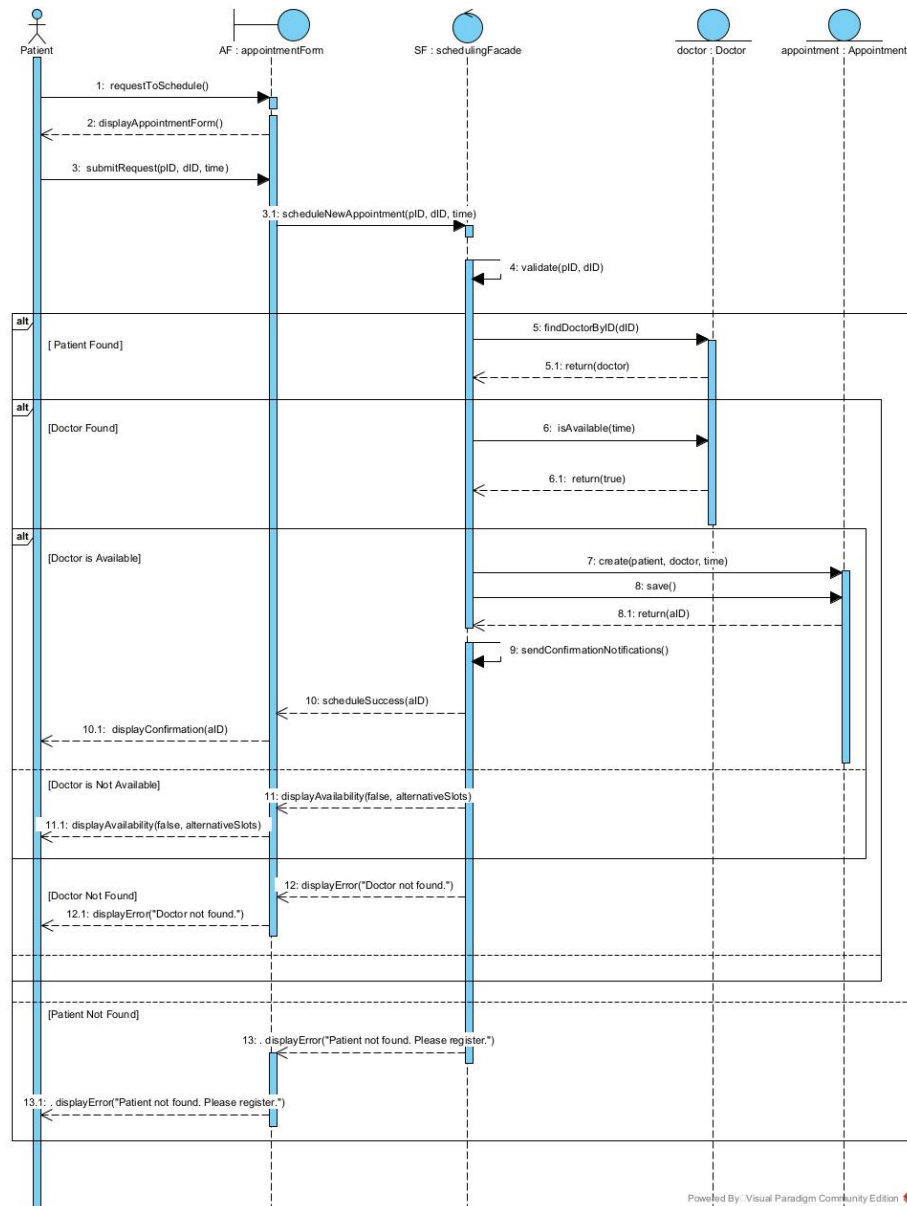


Diagram Description:

This diagram details the appointment scheduling workflow, managed by the SchedulingFacade. The design shows a strong validation sequence where the facade first validates the provided IDs (validate), then finds the Doctor object, and checks for availability (isAvailable). Upon successful validation, the facade uses a static factory method (create) to create a new Appointment object and then save it. This design effectively coordinates multiple entities (Doctor, Appointment) to accomplish a complex task while keeping the logic centralized within the facade.

Sprint 3: Patient Payment Processing

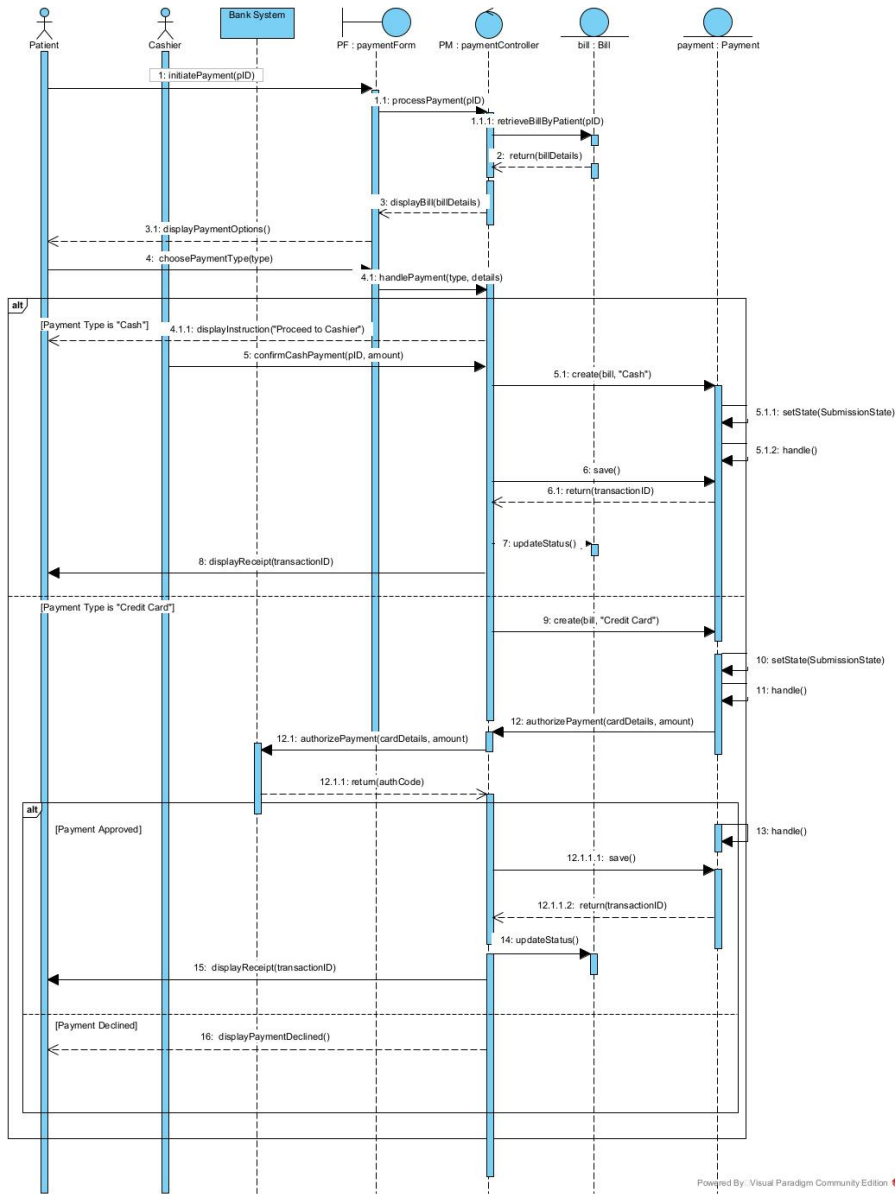


Diagram Description:

This diagram showcases the comprehensive payment process, handling both cash and credit card scenarios. The PaymentController retrieves the patient's bill and then manages the payment flow based on the selected type. A key feature of this design is the implementation of the State Design Pattern. The Payment entity's behavior is managed through `setState()` and `handle()` calls, allowing it to transition through different states, for example Submission, Processing and Approval without complex conditional logic in the Payment class itself. The diagram also correctly models the interaction with the external Bank System for credit card authorization.

Final cut class diagram

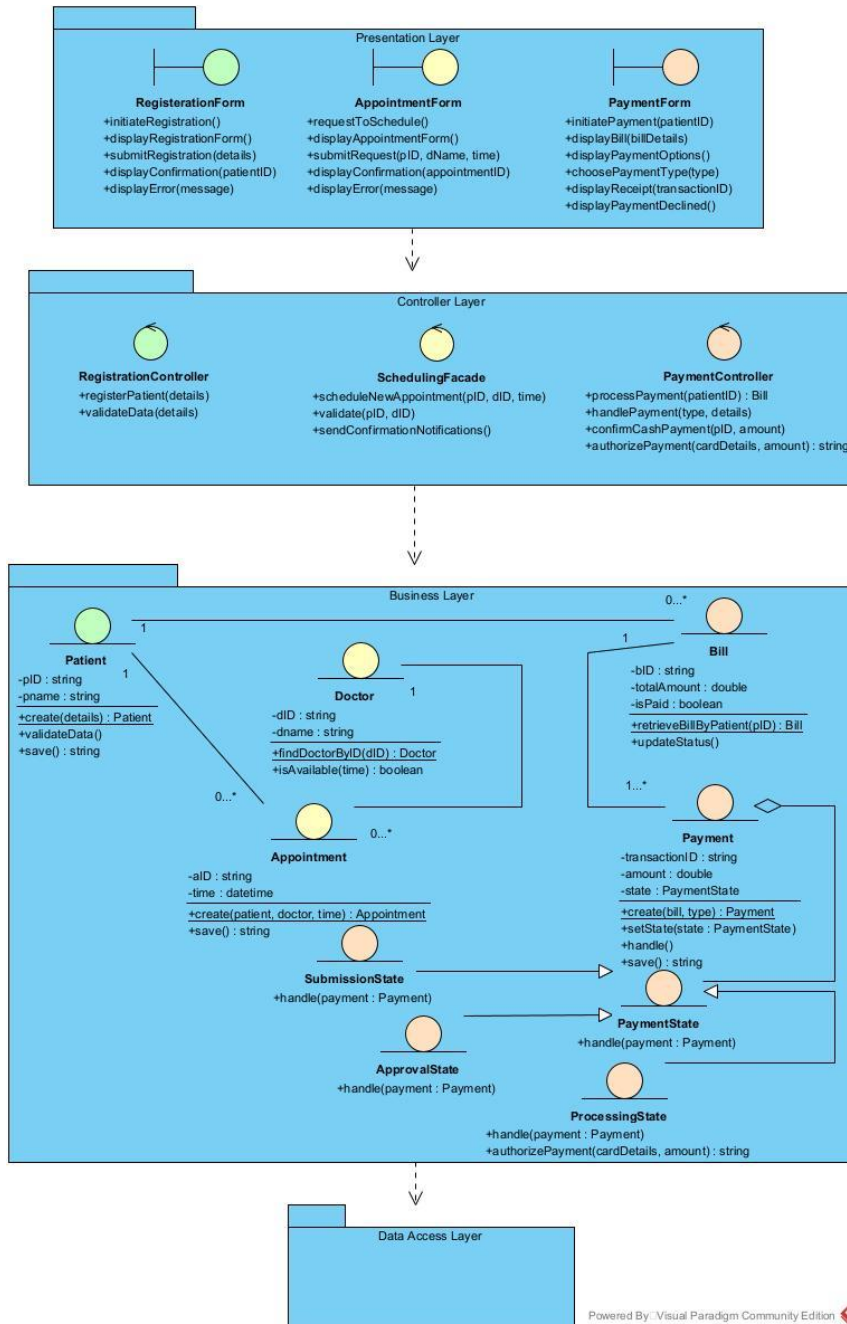


Diagram Description and Consistency Check:

This final class diagram is the blueprint for our system. We have made sure it is **consistent** with our sequence diagrams by checking the following points:

- **Existence:** Every object shown in the sequence of diagrams (like **RegistrationController**, **Patient**, and **Bill**) exists as a class in this diagram.

- **Visibility (Operations):** Every action (method call) from the sequence diagrams (like registerPatient(), isAvailable() and save()) is listed as an operation in the correct class. This proves that the class diagram supports the required actions.
- **Multiplicity:** The relationships between classes are correct. For example, the diagram shows that one Patient can have many (0..*) Appointments, which matches the logic in our sequence diagrams.

This process ensures that our design is consistent and ready for implementation.

4: IMPLEMENTATION MODEL

Introduction

This section describes the implementation model for the Patient Management System. This model translates the logical design into a physical blueprint, detailing the software components, the hardware infrastructure, and the database schema. It includes the component diagram, deployment diagram, and the entity-relationship diagram (ERD), which collectively provide a comprehensive guide for the system's development and deployment.

Component Diagram

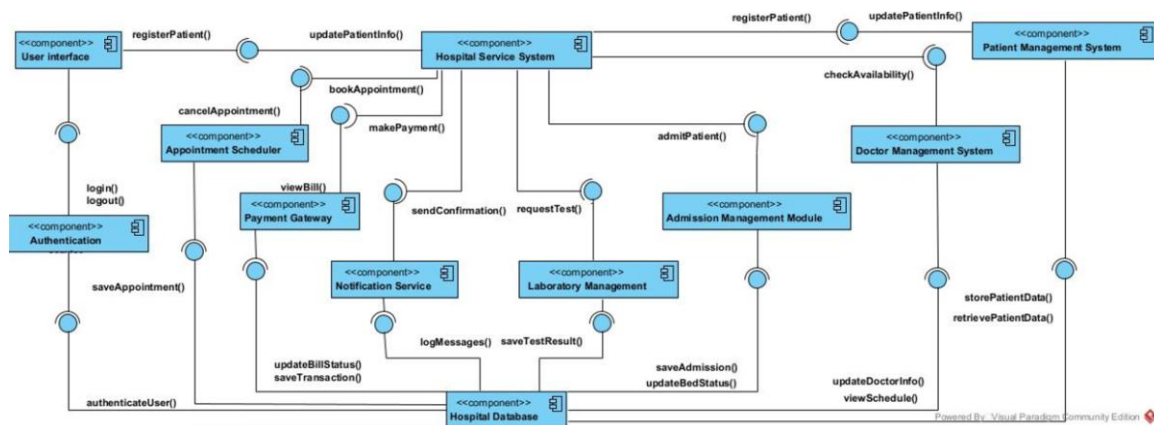


Diagram Description:

The component diagram illustrates the modular structure of the system, organized as a set of independent, service-oriented components. Each component encapsulates a specific business function, promoting reusability and maintainability. Key components include the Patient Management System, Appointment Scheduler, and Billing and Payment Module. These components interact through well-defined interfaces (e.g., `IPatientManagement`, `IAppointmentScheduling`), ensuring a decoupled and scalable architecture. The User Interface acts as the entry point, consuming the services provided by the backend components.

Deployment Diagram

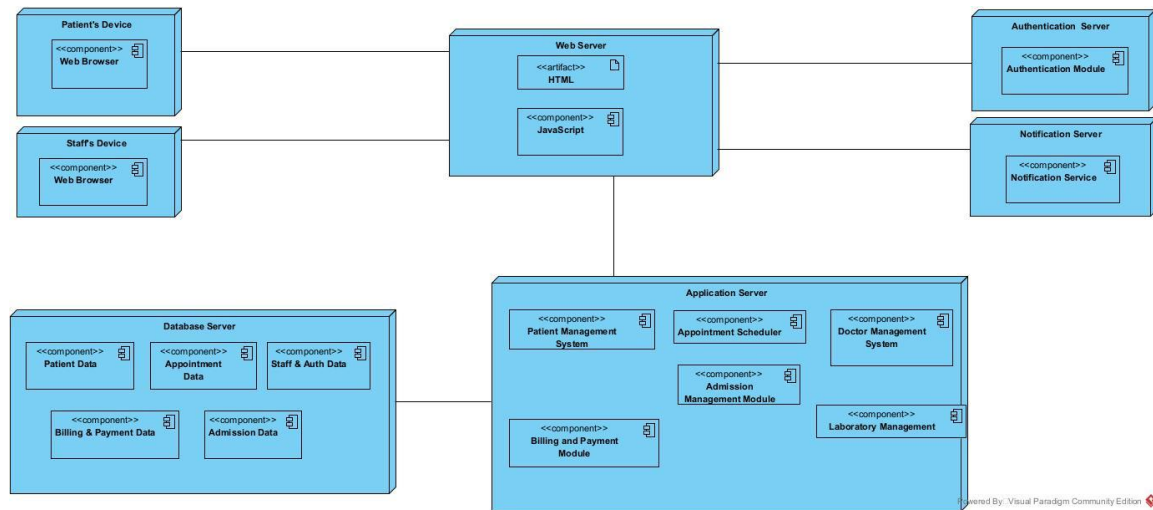


Diagram Description: The deployment diagram maps the software components to the physical hardware infrastructure, showcasing a distributed, multi-tier architecture. The system is deployed across several specialized servers to ensure security, scalability, and separation of concerns:

- **Web Server:** Hosts the static user interface files (HTML, JavaScript).
- **Application Server:** Runs the core business logic modules.
- **Database Server:** Manages and stores the system's data, organized into logical data components.
- **Authentication & Notification Servers:** Handle their respective tasks independently, which is a common practice in modern, secure systems.

This distributed setup illustrates how the system would operate in a real-world production environment.

Entity-Relationship Diagram (ERD)

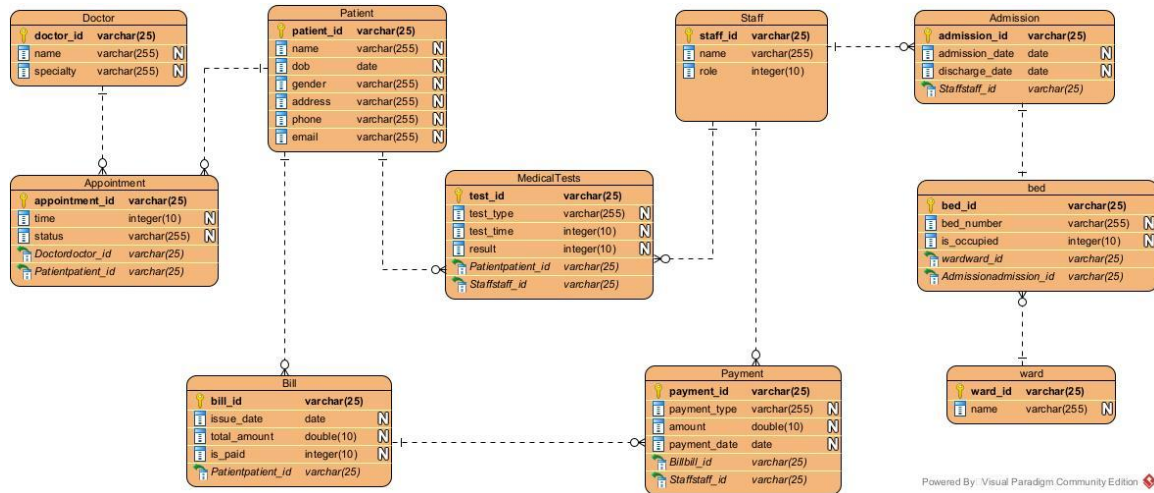


Diagram Description: The Entity-Relationship Diagram (ERD) provides the detailed schema for the hospital's database. It translates the entity classes from the design model into relational database tables. Each table is defined with its columns, primary keys (PK) for unique identification, and foreign keys (FK) to establish relationships between tables. For instance, the Appointments table uses foreign keys to link to the Patients and Doctors tables. This schema is optimized to ensure data integrity, minimize redundancy, and support all the functional requirements of the system.

Database Schema

The following database schema was derived from the Entity-Relationship Diagram (ERD). Each entity is mapped to a table, with attributes becoming columns. Primary Keys (PK) are identified to uniquely identify each record, and Foreign Keys (FK) are used to establish relationships between the tables.

- **Patient** (*patient_id*(PK), name, dob, gender, address, phone, email)
- **Doctor** (*doctor_id* (PK), name, specialty)
- **Staff** (*staff_id* (PK), name, role, username, password_hash)
- **Appointment** (*appointment_id*(PK), time, status, *doctor_id* (FK), *patient_id* (FK))
- **Bill** (*bill_id* (PK), issue_date, total_amount, is_paid, *patient_id* (FK))

- **Payment** (*payment_id* (PK), *payment_type*, *amount*, *payment_date*, *bill_id* (FK), *staff_id* (FK))
- **MedicalTests** (*test_id* (PK), *test_type*, *test_time*, *result*, *patient_id* (FK), *staff_id* (FK))
- **Admission** (*admission_id*(PK), *admission_date*, *discharge_date*, *staff_id* (FK))
- **Ward** (*ward_id*(PK), *name*)
- **Bed** (*bed_id* (PK), *bed_number*, *is_occupied*, *ward_id* (FK), *admission_id* (FK))

SQL Seed Script

This section provides the SQL CREATE TABLE script generated from the ERD. This script can be executed to create the physical database structure, including all tables, columns, data types, and primary/foreign key constraints.

```
CREATE TABLE Admission (
```

```
    admission_id varchar2(25) NOT NULL,
```

```
    admission_date date,
```

```
    discharge_date date,
```

```
    Staffstaff_id varchar2(25) NOT NULL,
```

```
    PRIMARY KEY (admission_id));
```

```
CREATE TABLE Appointment (
```

```
    appointment_id varchar2(25) NOT NULL,
```

```
    time          number(10),
```

```
    status        varchar2(255),
```

```
    Doctordoctor_id varchar2(25) NOT NULL,
```

```
    Patientpatient_id varchar2(25) NOT NULL,
```

```
    PRIMARY KEY (appointment_id));
```

```
CREATE TABLE bed (
```

```
    bed_id        varchar2(25) NOT NULL,
```

```
bed_number      varchar2(255),
is_occupied     number(10),
wardward_id     varchar2(25) NOT NULL,
Admissionadmission_id varchar2(25) NOT NULL,
PRIMARY KEY (bed_id));
```

CREATE TABLE Bill (

```
bill_id         varchar2(25) NOT NULL,
issue_date      date,
total_amount    double precision,
is_paid         number(10),
Patientpatient_id varchar2(25) NOT NULL,
PRIMARY KEY (bill_id));
```

CREATE TABLE Doctor (

```
doctor_id varchar2(25) NOT NULL,
name      varchar2(255),
specialty varchar2(255),
PRIMARY KEY (doctor_id));
```

CREATE TABLE MedicalTests (

```
test_id      varchar2(25) NOT NULL,
test_type    varchar2(255),
test_time    number(10),
result       number(10),
Patientpatient_id varchar2(25) NOT NULL,
Staffstaff_id  varchar2(25) NOT NULL,
PRIMARY KEY (test_id));
```

CREATE TABLE Patient (


```
patient_id varchar2(25) NOT NULL,  
name    varchar2(255),  
dob     date,  
gender  varchar2(255),  
address varchar2(255),  
phone   varchar2(255),  
email   varchar2(255),  
PRIMARY KEY (patient_id));  
  
CREATE TABLE Payment (  
    payment_id varchar2(25) NOT NULL,  
    payment_type varchar2(255),  
    amount      double precision,  
    payment_date date,  
    Billbill_id varchar2(25) NOT NULL,  
    Staffstaff_id varchar2(25) NOT NULL,  
    PRIMARY KEY (payment_id));  
  
CREATE TABLE Staff (  
    staff_id varchar2(25) NOT NULL,  
    name     varchar2(255),  
    role     number(10),  
    PRIMARY KEY (staff_id));  
  
CREATE TABLE ward (  
    ward_id varchar2(25) NOT NULL,  
    name     varchar2(255),  
    PRIMARY KEY (ward_id));
```

```
ALTER TABLE bed ADD CONSTRAINT FKbed576942 FOREIGN KEY  
(Admissionadmission_id) REFERENCES Admission (admission_id);
```

```
ALTER TABLE bed ADD CONSTRAINT FKbed776105 FOREIGN KEY (wardward_id)  
REFERENCES ward (ward_id);
```

```
ALTER TABLE Admission ADD CONSTRAINT FKAdmission612604 FOREIGN KEY  
(Staffstaff_id) REFERENCES Staff (staff_id);
```

```
ALTER TABLE Payment ADD CONSTRAINT FKPayment961605 FOREIGN KEY  
(Staffstaff_id) REFERENCES Staff (staff_id);
```

```
ALTER TABLE "MedicalTests"
```

```
ADD CONSTRAINT FKMedicalTes985401 FOREIGN KEY (Staffstaff_id) REFERENCES  
Staff (staff_id);
```

```
ALTER TABLE "MedicalTests"
```

```
ADD CONSTRAINT FKMedicalTes232046 FOREIGN KEY (Patientpatient_id)  
REFERENCES Patient (patient_id);
```

```
ALTER TABLE Payment ADD CONSTRAINT FKPayment978950 FOREIGN KEY  
(Billbill_id) REFERENCES Bill (bill_id);
```

```
ALTER TABLE Bill ADD CONSTRAINT FKBill812710 FOREIGN KEY (Patientpatient_id)  
REFERENCES Patient (patient_id);
```

```
ALTER TABLE Appointment ADD CONSTRAINT FKAppointment615677 FOREIGN KEY  
(Patientpatient_id) REFERENCES Patient (patient_id);
```

```
ALTER TABLE Appointment ADD CONSTRAINT FKAppointment836596 FOREIGN KEY  
(Doctordoctor_id) REFERENCES Doctor (doctor_id);
```

Java Skeleton Code

This section presents the Java skeleton code generated from the final design class diagram. Each class is represented as a .java file with its attributes and method signatures, providing a ready-to-use foundation for the development team to start implementing the business logic.

Patient.java

```
public class Patient {  
    private String patient_id;  
    private String name;  
    private Date dob;  
    private String gender;  
    private String address;  
    private String phone;  
    private String email;  
  
    private List<Appointment> appointments = new ArrayList<>(  
  
);  
  
    private List<Bill> bills = new ArrayList<>(  
  
);  
    private List<MedicalTests> medicalTests = new ArrayList<>(  
  
);  
}
```

Doctor.java

```
public class Doctor {  
    private String doctor_id;  
    private String name;  
    private String specialty;  
  
    private List<Appointment> appointments = new ArrayList<>();  
}
```

Staff.java

```
public class Staff {  
    private String staff_id;  
    private String name;  
    private int role;  
  
    private List<Payment> payments = new ArrayList<>();  
  
    private List<Admission> admissions = new ArrayList<>();  
  
    private List<MedicalTests> medicalTests = new ArrayList<>();  
}
```

Appointment.java

```
public class Appointment {  
    private String appointment_id;  
    private int time;  
    private String status;  
  
    private Doctor doctor;  
  
    private Patient patient;  
}
```

Bill.java

```
public class Bill {  
    private String bill_id;  
    private Date issue_date;  
    private double total_amount;  
    private int is_paid;  
  
    private Patient patient;  
  
    private List<Payment> payments = new ArrayList<>(  
    );  
}
```

Payment.java

```
public class Payment {  
    private String payment_id;  
    private String payment_type;  
    private double amount;  
    private Date payment_date;  
  
    private Bill bill;  
  
    private Staff staff;  
}
```

MedicalTests.java

```
public class MedicalTests {  
    private String test_id;  
    private String test_type;  
    private int test_time;  
    private int result;  
  
    private Patient patient;  
  
    private Staff staff;  
}
```

Admission.java

```
public class Admission {  
    private String admission_id;  
    private Date admission_date;  
    private Date discharge_date;  
    private Staff staff;  
    private Bed bed;  
}
```

File: Ward.java

```
public class Ward {  
    private String ward_id;  
    private String name;  
  
    private List<Bed> beds = new ArrayList<>(  
  
);  
  
}
```

Bed.java

```
public class Bed {  
    private String bed_id;  
    private String bed_number;  
    private int is_occupied;  
    private Ward ward;  
    private Admission admission;  
  
}
```

▼ C5PMS Sprint 3 16 Nov – 19 Nov (6 work items) 0 0 0 Complete sprint ...		
☐	☒ G5PMS-16	Implement Component Diagram
☐	☒ G5PMS-17	Implement Deployment Diagram
☐	☒ G5PMS-18	Implement Entity Relation Diagram
☐	☒ G5PMS-19	Generate The Database Schema
☐	☒ G5PMS-20	Create The SQL query (seed script)
☐	☒ G5PMS-22	Create The Java skeleton

Evidence of collaboration full project

Spaces

 **Case 5: Patient Management System** 

 Summary
  Timeline
  Backlog
  Board
  Calendar
  List
  Forms
  Goals
  All work
  Development

 Ask AI
  Basic
  JQL

 Project
  Case 5: Patient Management System
  Assignee
  Type
  Status

☐	Work	Assignee	Reporter	Status
☐	  ESPMS-22 Create The Java skeleton	 Maryam Ali	 Maryam Ali	 DONE 
☐	  ESPMS-20 Create The SQL query (seed script)	 Hk Bd	 Maryam Ali	 DONE 
☐	  ESPMS-19 Generate The Database Schema	 Maryam Ali	 Maryam Ali	 DONE 
☐	  ESPMS-18 Implement Entity Relation Diagram	 Hk Bd	 Maryam Ali	 DONE 
☐	  ESPMS-17 Implement Deployment Diagram	 Hk Bd	 Maryam Ali	 DONE 
☐	  ESPMS-16 Implement Component Diagram	 Maryam Ali	 Maryam Ali	 DONE 
☐	  ESPMS-15 Task 2: Create Design Sequence Diagram (...)	 Maryam Ali	 Maryam Ali	 DONE 
☐	  ESPMS-14 Detail Use Use Case #3 (Textual Descripti...	 Unassigned	 Hk Bd	 DONE 
☐	  ESPMS-12 Task 2.4 (Joint): Verify Consistency betwe...	 Unassigned	 Hk Bd	 DONE 
☐	  ESPMS-11 Task 2.3 (Joint): Create Design Sequence ...	 Unassigned	 Hk Bd	 DONE 
☐	  ESPMS-10 Task 2.2 (Student 2): Create Design Sequen...	 Hk Bd	 Hk Bd	 DONE 
☐	  ESPMS-9 Task 2.1 (Student 1): Create Design Sequen...	 Maryam Ali	 Hk Bd	 DONE 
☐	  ESPMS-8 Design High-Level System Architecture (do...	 Unassigned	 Hk Bd	 DONE 
☐	  ESPMS-7 Refine Class Diagram with Design Patterns	 Hk Bd	 Hk Bd	 DONE 
☐	  ESPMS-6 Refine Class Diagram with Design Patterns	 Maryam Ali	 Hk Bd	 DONE 
☐	  ESPMS-5 Create & Integrate Domain Class Diagram (...)	 Unassigned	 Hk Bd	 DONE 
☐	  ESPMS-4 Detail Use Case #2 (Textual Description & S...	 Hk Bd	 Hk Bd	 DONE 
☐	  ESPMS-3 Detail Use Case #1 (Textual Description & S...	 Maryam Ali	 Hk Bd	 DONE 
☐	  ESPMS-2 Create Main Use Case Diagram (done by bo...	 Unassigned	 Hk Bd	 DONE 
☐	  ESPMS-1 Define Functional & Non-Functional Require...	 Unassigned	 Hk Bd	 DONE 